



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ingeniería Eléctrica

DSOGI and MSOGI Module Documentation

Disclaimer: The digital implementation of the control structures detailed in this document is modeled after the Dual Second-Order Generalized Integrator (DSOGI) framework outlined by Imperix [2].

1. DSOGI Module (`dsogi.h` / `dsogi.c`)

1.1. The Second-Order Generalized Integrator (SOGI)

The **Second-Order Generalized Integrator** (SOGI) is a frequency-adaptive resonant filter built around a closed loop of two integrators tuned to a prescribed frequency ω . Like any resonant system, the loop responds strongly to excitation at its resonant frequency and weakly everywhere else: when a distorted signal is applied at its input, the internal states naturally lock onto and **reproduce the component oscillating at ω , while harmonics, noise, and offsets are progressively attenuated**. The SOGI therefore behaves as a selective band-pass filter whose centre frequency is a run-time parameter rather than a fixed design constant — it can simply be told, at every step, which frequency to extract.

The resonant structure provides a second, equally valuable property at no extra cost: because one integrator state is the integral of the other, the loop inherently maintains a copy of the extracted component shifted by exactly 90° . The SOGI then acts as an *Orthogonal Signal Generator* (OSG), delivering an in-phase/quadrature pair from a single scalar measurement. This pair is precisely the ingredient needed to decompose an unbalanced three-phase system into its symmetrical sequence components and, from there, to synchronise a converter to the grid — which is why the SOGI is the elementary building block of the DSOGI and MSOGI-PLL structures studied throughout this document. Both properties — selective extraction and orthogonal signal generation — are formalised by the transfer functions below and exploited directly in the positive-sequence extraction stage described in Section 1.5.

The filter is characterised in the Laplace domain by two transfer functions relating the input $U(s)$ to the in-phase output $V(s)$ and the quadrature output $QV(s)$:

$$D(s) = \frac{V(s)}{U(s)} = \frac{k\omega s}{s^2 + k\omega s + \omega^2} \quad (1)$$

$$Q(s) = \frac{QV(s)}{U(s)} = \frac{k\omega^2}{s^2 + k\omega s + \omega^2} \quad (2)$$

where $k > 0$ is the damping gain and ω is the resonant (centre) frequency. Equation (1) defines a second-order bandpass response centred at ω , while equation (2) defines a second-order low-pass response. At the resonant frequency, both transfer functions have unity magnitude, and their phase relationship satisfies $\angle Q(j\omega) - \angle D(j\omega) = -90$; that is, QV lags V by exactly 90 under ideal, steady-state conditions.

1.2. State-Space Realisation

To implement the SOGI on a digital processor, the transfer functions (1)–(2) must be cast into a set of first-order differential equations for numerical integration. Selecting v and qv as state variables and applying the inverse Laplace transform, the state-space realisation adopted in `dsogi.c` is:

$$\dot{v} = k\omega(u - v) + \omega qv \quad (3)$$

$$\dot{qv} = -\omega v \quad (4)$$

It is important to note that this implementation corresponds to the *negative* of the standard quadrature output. Consequently, qv in this code represents a +90 phase lead relative to v (i.e., -90 lag expressed with opposite sign), a convention that propagates into the positive-sequence extraction formulas of Section 1.5.

The frequency-adaptive nature of the filter is made explicit by equations (3)–(4): the **resonant frequency ω appears as a parameter that can be updated at every time step, allowing the filter to track grid frequency variations in real time without any coefficient recalculation.**

1.3. Dual SOGI (DSOGI): Extension to the $\alpha\beta$ Frame

A single SOGI processes a scalar input. After the Clarke transformation, grid voltages or currents are represented as an (α, β) pair in the stationary reference frame. The **Dual SOGI (DSOGI)** handles this by applying two structurally identical and *independent* SOGI instances in parallel—one for the α axis and one for the β axis—sharing the same frequency ω and gain k . The resulting system has four state variables:

$$\dot{v}_\alpha = k\omega(v_{\alpha,\text{in}} - v_\alpha) + \omega qv_\alpha \quad (5)$$

$$\dot{qv}_\alpha = -\omega v_\alpha \quad (6)$$

$$\dot{v}_\beta = k\omega(v_{\beta,\text{in}} - v_\beta) + \omega qv_\beta \quad (7)$$

$$\dot{qv}_\beta = -\omega v_\beta \quad (8)$$

1.4. Input Error Saturation

Before computing the state derivatives, the tracking error on each axis is clamped to a maximum value $E_{\text{máx}}$:

$$e_\alpha = \text{sat}_{E_{\text{m}\acute{\text{a}}\text{x}}}(v_{\alpha,\text{in}} - v_\alpha), \quad e_\beta = \text{sat}_{E_{\text{m}\acute{\text{a}}\text{x}}}(v_{\beta,\text{in}} - v_\beta) \quad (9)$$

This saturation prevents the integrators from winding up during start-up or sudden phase jumps, bounding the rate of state change and ensuring numerical stability without altering the small-signal dynamics at steady state.

1.5. Symmetrical Sequence Extraction

The fundamental motivation for processing α and β simultaneously is to enable the extraction of symmetrical sequence components directly in the stationary frame. In a general unbalanced three-phase system, the $\alpha\beta$ voltage vector can be decomposed into a positive-sequence component rotating at $+\omega$ and a negative-sequence component rotating at $-\omega$. The DSOGI's quadrature states provide the 90 phase-shift operator required to algebraically separate these counter-rotating vectors.

Accounting for the sign convention established in Section 1.2—where qv in this implementation represents -90 lag with opposite sign, i.e., a $+90$ lead—the positive- and negative-sequence components in the $\alpha\beta$ frame are given by:

$$v_\alpha^+ = \frac{1}{2}(v_\alpha + qv_\beta) \quad (10)$$

$$v_\beta^+ = \frac{1}{2}(v_\beta - qv_\alpha) \quad (11)$$

$$v_\alpha^- = \frac{1}{2}(v_\alpha - qv_\beta) \quad (12)$$

$$v_\beta^- = \frac{1}{2}(v_\beta + qv_\alpha) \quad (13)$$

Equations (10)–(13) are implemented in the dedicated **Sequence_Separator** module (`sequence_separator.c`), which operates on the states of the fundamental DSOGI instance ($h = 1$). The additive combination cancels the contribution of the negative-sequence vector on the positive-sequence estimate, and vice versa, providing clean sequence separation even under severe grid unbalance. Both the positive-sequence pair (10)–(11) and the negative-sequence pair (12)–(13) are computed at every step and exported by the MSOGI-PLL module.

1.6. Numerical Integration: Adams-Bashforth 3 (AB3)

The four state equations are integrated using the **Adams-Bashforth 3-step** (AB3) explicit multistep method. The AB3 update rule is:

$$x[n+1] = x[n] + \frac{T_s}{12}(23\dot{x}[n] - 16\dot{x}[n-1] + 5\dot{x}[n-2]) \quad (14)$$

AB3 is third-order accurate in the step size T_s , offering significantly lower truncation error than forward Euler ($\mathcal{O}(T_s)$) or the trapezoidal method ($\mathcal{O}(T_s^2)$), at the cost of requiring two past derivative values per state. Since these past values are unavailable during the first two steps, the implementation employs a self-starting sequence:

step_count	Method	Formula
0	Forward Euler	$x[1] = x[0] + T_s \dot{x}[0]$
1	Adams-Bashforth 2	$x[2] = x[1] + \frac{T_s}{2}(3\dot{x}[1] - \dot{x}[0])$
≥ 2	Adams-Bashforth 3	Equation (14)

Each of the four states maintains its own independent derivative history buffer (`dv_alpha_hist`, `dqv_alpha_hist`, `dv_beta_hist`, `dqv_beta_hist`), so the startup ramp does not introduce cross-axis interference and all states share the same `step_count` counter, which is incremented once per control cycle.

2. MSOGI-PLL Module (`msogi_pll.h` / `msogi_pll.c`)

2.1. Architecture Overview

The **Multiple Second-Order Generalized Integrator Phase-Locked Loop** (MSOGI-PLL) is a grid synchronisation algorithm designed to operate robustly under distorted and unbalanced grid conditions. It extends the classical SOGI-PLL by deploying a bank of up to `MAX_HARMONICS` = 8 DSOGI instances, each tuned to a different harmonic order of the estimated fundamental frequency, and interconnected through a Cross-Decoupling Network (CDN) that enforces selective harmonic separation. In this project the bank is configured with harmonic orders $\{1, 5, 7\}$, targeting the dominant low-order harmonics of a typical three-phase grid.

Each stage is described in the following subsections.

2.2. Clarke Transformation Stage

The three-phase voltage measurements (V_a, V_b, V_c) are first mapped to the stationary α - β -0 frame using the amplitude-invariant Clarke transformation. The resulting v_α and v_β serve as inputs to the MSOGI bank. The zero-sequence component v_0 is retained in the structure for completeness but does not participate in the synchronisation loop, as the positive-sequence extraction operates entirely in the $\alpha\beta$ plane.

2.3. MSOGI Bank with Cross-Decoupling Network (CDN)

2.3.1. Motivation

A single DSOGI tuned to the fundamental frequency ω_1 will pass not only the fundamental component but also attenuated replicas of any harmonics present in the input, due to its finite bandwidth. To achieve selective and simultaneous extraction of multiple harmonic components, the module employs a **Cross-Decoupling Network** (CDN), which forms the architectural core of the MSOGI structure.

2.3.2. CDN Principle

Let N DSOGI instances be indexed $i = 1, \dots, N$, each tuned to harmonic order h_i at frequency $\omega_{h_i} = h_i \omega$. Denote the output of the i -th instance as $(\hat{v}_\alpha^{(i)}, \hat{v}_\beta^{(i)})$. In

steady state, the total reconstructed signal is the superposition $\hat{v}_\alpha = \sum_i \hat{v}_\alpha^{(i)}$, i.e., each DSOGI contributes only its own harmonic component to the aggregate.

To enforce this separation, the CDN feeds each DSOGI a *residual* input formed by subtracting the contributions of all other instances from the raw Clarke output:

$$v_{\alpha,\text{in}}^{(i)} = v_\alpha - \sum_{j \neq i} \hat{v}_\alpha^{(j)} \quad (15)$$

$$v_{\beta,\text{in}}^{(i)} = v_\beta - \sum_{j \neq i} \hat{v}_\beta^{(j)} \quad (16)$$

Intuitively, each DSOGI sees the full input signal with the estimated contributions of all other harmonics already removed. The CDN therefore creates a set of N coupled adaptive notch filters that converge iteratively to the harmonic decomposition of the input: in steady state, $\hat{v}_\alpha^{(i)} \rightarrow v_\alpha^{(h_i)}$, the true h_i -th harmonic component of v_α .

It is worth noting that the CDN relies on the previous time step's DSOGI outputs to form the residuals: before any instance is updated, the outputs of all DSOGIs are copied into a frozen snapshot, and every residual in (15)–(16) is computed from that snapshot. **This guarantees that the decoupling is independent of the order in which the bank is updated.** The explicit one-step lag it introduces does not affect steady-state accuracy and is a standard implementation trade-off in discrete-time MSOGI realisations.

2.3.3. Per-Harmonic Gain Scaling

For the i -th harmonic of order h_i , the DSOGI tuning frequency and damping gain are set as:

$$\omega_{h_i} = h_i \omega, \quad k_{h_i} = \frac{k_{\text{sogi}}}{h_i} \quad (17)$$

The inverse scaling of k with harmonic order compensates for the proportional increase in resonant frequency. Since the SOGI bandwidth is approximately $k_{h_i} \omega_{h_i} = k_{\text{sogi}} \omega$ (constant), this scaling maintains a uniform absolute bandwidth in rad/s across all harmonic channels, preventing excessive noise amplification at high orders while preserving comparable transient response times.

2.4. Positive-Sequence Extraction

Once the MSOGI bank has converged, only the fundamental DSOGI ($h_1 = 1$) is used for sequence extraction, since the outer PLL loop requires a single, clean rotating vector at ω . The extraction is carried out by the dedicated **Sequence_Separator** module, which relies on the quadrature states of the first DSOGI instance and the sign convention established in Section 1.2: in this implementation qv leads v by 90, so that at steady state, for a balanced positive-sequence input, the states satisfy:

$$qv_\alpha^{(1)} \approx -v_\beta, \quad qv_\beta^{(1)} \approx v_\alpha \quad (18)$$

Using these relations, the positive- and negative-sequence $\alpha\beta$ components are recovered algebraically as:

$$v_{\alpha}^{+} = \frac{1}{2} \left(v_{\alpha}^{(1)} + qv_{\beta}^{(1)} \right) \quad (19)$$

$$v_{\beta}^{+} = \frac{1}{2} \left(v_{\beta}^{(1)} - qv_{\alpha}^{(1)} \right) \quad (20)$$

The validity of (19)–(20) can be verified by substituting (18): each expression doubles the positive-sequence contribution while cancelling the negative-sequence term, which rotates in the opposite direction and therefore produces quadrature states of opposite sign. This algebraic cancellation provides inherent negative-sequence rejection without requiring any additional notch filter or double-frequency compensation, which is the principal advantage of the DSOGI-based approach over classical SRF-PLLs under unbalanced grid conditions. The complementary combinations yield the negative-sequence pair $(v_{\alpha}^{-}, v_{\beta}^{-})$, which the module computes simultaneously and exposes through the `v_alpha_neg` / `v_beta_neg` fields of the sequence-separator sub-structure.

2.5. Park Transformation and Error Signal Generation

The positive-sequence $\alpha\beta$ vector is projected onto the estimated synchronous dq frame via the Park transformation at angle $\hat{\theta}$:

$$\begin{bmatrix} v_d \\ v_q \end{bmatrix} = \begin{bmatrix} \cos \hat{\theta} & \sin \hat{\theta} \\ -\sin \hat{\theta} & \cos \hat{\theta} \end{bmatrix} \begin{bmatrix} v_{\alpha}^{+} \\ v_{\beta}^{+} \end{bmatrix} \quad (21)$$

When the PLL is locked and $\hat{\theta}$ is aligned with the positive-sequence voltage vector angle, $v_d = |\mathbf{v}^{+}|$ and $v_q = 0$. A phase error $\Delta\phi = \hat{\theta} - \theta_{\text{grid}}$ produces $v_q \approx |\mathbf{v}^{+}| \sin(\Delta\phi) \approx |\mathbf{v}^{+}| \Delta\phi$ for small errors, making v_q a natural phase error signal.

To decouple the loop dynamics from grid voltage amplitude variations—which would otherwise change the effective PI gain— v_q is normalised by the instantaneous amplitude of the positive-sequence vector:

$$\varepsilon = \frac{v_q}{|\mathbf{v}^{+}|}, \quad |\mathbf{v}^{+}| = \sqrt{(v_{\alpha}^{+})^2 + (v_{\beta}^{+})^2} \quad (22)$$

This normalisation renders the loop bandwidth independent of the grid voltage level. In the implementation, the amplitude estimate is bounded below at 10 V before the division, preventing numerical blow-up during the first instants of start-up when the sequence estimates are still close to zero.

The normalised error is subsequently smoothed by a fixed first-order IIR filter:

$$\varepsilon_f[n] = 0.98 \varepsilon_f[n-1] + 0.02 \varepsilon[n] \quad (23)$$

The filter coefficient 0.98 corresponds to a time constant of $\tau = -T_s / \ln(0.98) \approx 4.95$ ms for $T_s = 100$ μ s, providing effective suppression of double-frequency ripple (at 100 Hz) introduced by residual negative-sequence content, without adding significant phase lag to the outer PLL loop.

2.6. PI Frequency Controller

The filtered error ε_f drives a discrete PI controller that estimates the angular frequency deviation from the nominal value:

$$\hat{\omega}[n] = \omega_{\text{nom}} + \delta\omega[n], \quad \delta\omega[n] = \mathcal{C}_{\text{PI}}(\varepsilon_f[n]) \quad (24)$$

where $\omega_{\text{nom}} = 2\pi f_{\text{nom}}$ is derived from the nominal grid frequency passed at initialisation ($f_{\text{nom}} = 50$ Hz in this project) rather than being hardcoded. The PI action provides zero steady-state error for a constant phase offset (integral term accumulates until $\varepsilon_f = 0$) and proportional correction for dynamic frequency deviations. The output is subject to an anti-windup saturation limit `pi_limit`, which bounds the maximum allowable frequency deviation and prevents integrator runaway during large transients.

2.7. Angle Integration

The angular frequency estimate is integrated to obtain the phase angle using the **trapezoidal rule**:

$$\hat{\theta}[n] = \hat{\theta}[n-1] + \frac{T_s}{2}(\hat{\omega}[n] + \hat{\omega}[n-1]) \quad (25)$$

The trapezoidal rule is second-order accurate in T_s , in contrast to forward Euler's first-order accuracy, reducing the phase accumulation error per step from $\mathcal{O}(T_s^2)$ to $\mathcal{O}(T_s^3)$. Since the PLL integrates continuously over many thousands of cycles, this improvement in per-step accuracy translates into a substantially lower steady-state phase error in the estimated angle.

2.8. Start-Up Phase Trim

After the trapezoidal integration step, a supplementary **phase trim** is applied, but only during start-up (It's not required in a real micro-controller implementation given the fact that it uses the `atan2` function which increases the resources required for the implementation). The angle of the positive-sequence vector provides a direct, model-free estimate of the grid phase angle:

$$\hat{\theta}_{\text{inst}} = \text{atan2}(v_{\beta}^+, v_{\alpha}^+) \quad (26)$$

The discrepancy between this instantaneous estimate and the integrated angle defines a residual phase error:

$$\Delta\theta = \text{wrap}_{[-\pi, \pi]}(\hat{\theta}_{\text{inst}} - \hat{\theta}) \quad (27)$$

which, while the trim is active, is fed back with a small corrective gain:

$$\hat{\theta} \leftarrow \hat{\theta} + K_{\theta} \Delta\theta, \quad K_{\theta} = 0.01 \quad (28)$$

The angle wrapping in (27) ensures the correction is always taken through the shortest arc, avoiding 2π jumps.

The role of this trim is deliberately limited to accelerating the initial lock transient: being a direct angle correction, it pulls $\hat{\theta}$ toward the measured vector angle much faster than the PI loop alone, which shortens the settling portion of every simulation run. It is, however, *not* left running in steady state. A continuously active trim fights the PI controller’s own v_q -zeroing action: it prevents the filtered error ε_f from ever settling to true zero, so the integral term keeps accumulating and the frequency estimate drifts without bound until it rails at the saturation limit `pi_limit`. For this reason the trim is governed by the `trim_active` flag: it starts enabled and is latched off permanently the first time the PLL achieves lock (Section 2.12). The latch is one-way — the trim is never re-enabled, even if lock is subsequently lost — since the PI loop alone is fast enough once the system operates near its nominal point, and re-enabling the trim would only reopen the long-term drift risk. After all corrections are applied, $\hat{\theta}$ is wrapped to $[0, 2\pi)$ to maintain a canonical representation.

2.9. Angle Export Compensation

The angle exported to the rest of the control chain, `theta_out`, is not the raw internal state $\hat{\theta}$ but a slightly advanced copy:

$$\theta_{\text{out}} = \text{wrap}_{[0, 2\pi)}\left(\hat{\theta} + \hat{\omega}(T_s + T_{\text{conv}})\right), \quad T_{\text{conv}} = 2.273 \text{ ms} \quad (29)$$

The term $\hat{\omega}T_s$ compensates the one-sample delay between the instant at which the angle is computed and the instant at which its consumers apply it, while T_{conv} is an empirically measured, fixed offset that reconciles the PLL’s internal angle reference with the phase convention used by the rest of the chain. The internal state $\hat{\theta}$ itself is left untouched by this export step, so the compensation introduces no dynamics into the loop.

2.10. Start-Up Frequency Scheduling

During the initial transient, the DSOGI states are far from their steady-state values and the PI estimate $\hat{\omega}$ is unreliable. Driving the DSOGI bank at an incorrect frequency during this phase would delay or prevent convergence. To mitigate this, the frequency supplied to each DSOGI is scheduled as a convex combination of the nominal frequency and the PI estimate:

$$\omega_{\text{SOGI}}[n] = (1 - \alpha[n])\omega_{\text{nom}} + \alpha[n]\hat{\omega}[n] \quad (30)$$

where the blending coefficient $\alpha[n]$ ramps linearly from 0 to 1 according to:

$$\alpha[n] = \begin{cases} 0 & n < 100 \\ \frac{n - 100}{100} & 100 \leq n < 200 \\ 1 & n \geq 200 \end{cases} \quad (31)$$

For a sampling period $T_s = 100 \mu\text{s}$, the hold phase spans the first 10 ms of operation—allowing the DSOGI states to begin settling at the nominal frequency—followed by a 10 ms linear handover, so that closed-loop frequency tracking is fully

engaged from 20 ms onwards. This scheduled initialisation prevents the positive feedback that would occur if the PI loop were closed immediately with uninitialised DSOGI states. Note that the thresholds in (31) are hardcoded sample counts, so the durations quoted above scale accordingly if T_s is changed.

2.11. Output Frequency Filtering

The raw frequency estimate $\hat{\omega}[n]$, which inherits high-frequency content from the PI output, is passed through a configurable first-order low-pass filter before being exposed as the module output `omega_out`:

$$\hat{\omega}_f[n] = \text{CALCULATE_LP_FILTER_1D}(\hat{\omega}[n]) \quad (32)$$

The cutoff frequency `lpf_omega_fc` is set at initialisation. Crucially, the angle integrator (25) operates on the *unfiltered* $\hat{\omega}$, so no additional phase lag is introduced into $\hat{\theta}$. The filtered output `omega_out` is intended for monitoring and feed-forward purposes only.

2.12. Lock Detection and Trim Latch-Off

At every step the module evaluates a lock criterion, used both as a supervisory status flag and to drive the start-up sequencing. The PLL is declared locked when all three of the following conditions hold simultaneously for $N_{\text{lock}} = 200$ consecutive samples (about one grid cycle at $T_s = 100 \mu\text{s}$):

Condition	Constant	Interpretation
$ \mathbf{v}^+ > 50 \text{ V}$	<code>AMP_LOCK_MIN</code>	a live positive-sequence vector is present
$ \varepsilon_f < 0.05$	<code>VQ_LOCK_THR</code>	the normalised q -axis error has collapsed
$ \hat{\omega} - \omega_{\text{nom}} < 2\pi \cdot 1 \text{ rad/s}$	<code>W_LOCK_BAND</code>	frequency within $\pm 1 \text{ Hz}$ of nominal

Any violation resets the consecutive-sample counter to zero. The resulting `locked` flag is exported for supervisory use, and its first assertion irreversibly clears `trim_active`, disabling the start-up phase trim of Section 2.8 for the remainder of operation. Once the trim is latched off, ε_f converges to essentially zero, so the threshold `VQ_LOCK_THR` acts as a comfortable margin rather than a level sitting just above a permanent bias.

2.13. Summary of Module Outputs

Field	Symbol	Description
<code>theta_out</code>	θ_{out}	Exported grid angle [rad], $\in [0, 2\pi)$, incl. Eq. (29)
<code>omega_out</code>	$\hat{\omega}_f$	Low-pass filtered grid frequency [rad/s]
<code>v_d</code>	v_d	d -axis positive-sequence voltage [V]
<code>v_q</code>	v_q	q -axis positive-sequence voltage [V]
<code>v_alpha_pos</code> , <code>v_beta_pos</code>	$v_{\alpha,\beta}^+$	Positive-sequence stationary voltages [V]
<code>seq.v_alpha_neg</code> , <code>seq.v_beta_neg</code>	$v_{\alpha,\beta}^-$	Negative-sequence stationary voltages [V]
<code>v_zero</code>	v_0	Clarke zero-sequence component [V]
<code>locked</code>	—	Lock status flag (Section 2.12)

Referencias

- [1] Imperix. (2021). *SOGI PLL* (Technical Note TN104). Updated March 2026. Retrieved from <https://imperix.com/doc/implementation/sogi-pll?currentThread=single-phase-pv-inverter-with-fictive-axis-emulation>